УНИВЕРСИТЕТ СИНЕРГИЯ

АВТОНОМНАЯ НЕКОММЕРЧЕСКАЯ ОРГАНИЗАЦИЯ ВЫСШЕГО ОБРАЗОВАНИЯ «МОСКОВСКИЙ УНИВЕРСИТЕТ «СИНЕРГИЯ»

Факультет Информационных технологий
(наименование факультета/ института)

Направление подготовки /специальность: 09.03.03 Прикладная информатика
(код и наименование направления подготовки /специальности)

Профиль/специализация: Искусственный интеллект и большие данные
(наименование профиля/специализации)

ОТЧЕТ

ПО УЧЕБНОЙ ПРАКТИКЕ
(вид практики)

Технологическая (проектно-технологическая) практика
(тип практики)

Обучающийся Старцев Александр Владимирович _____
(Ф.И.О.) (подпись)

Москва 2026г.

Практические кейсы-задачи, необходимые для оценки умений, навыков и (или) опыта деятельности по итогам практики

№ п/п	Подробные ответы обучающегося на практические кейсы-задачи
Кейс-задача № 1	В рамках задачи разработана программа на языке Python, которая обрабатывает одномерный массив A размерности N и вычисляет сумму отрицательных элементов, расположенных строго между максимальным и минимальным элементами. Алгоритм сначала за один проход определяет индексы максимального и минимального элементов массива, после чего вычисляет границы интервала между ними как диапазон индексов от меньшего до большего из найденных индексов. Сами крайние элементы (максимум и минимум) в интервал не включаются, что соответствует формулировке «между». Далее программа суммирует только те элементы интервала, значение которых меньше нуля. Решение имеет линейную вычислительную сложность O(N) и не требует дополнительной памяти, поэтому эффективно работает даже на больших массивах. Отдельно обработаны граничные случаи: пустой массив, массив длиной менее трёх элементов и ситуация, когда максимум и минимум стоят рядом — в этих случаях между крайними элементами ничего нет и результат равен нулю. Для проверки корректности написан набор автоматических тестов (test_task1.py), который покрывает как типовые, так и граничные сценарии и проходит успешно. Например, для массива [1, 9, -3, 4, -1, -7, 2] максимум 9 (индекс 1) и минимум -7 (индекс 5), между ними находятся элементы -3, 4, -1, а сумма отрицательных равна -4. Программа снабжена интерактивным вводом и подробными комментариями, что упрощает её сопровождение и повторное использование. Ссылка на GitHub: https://github.com/StarcevAlexander/synergy-practice/tree/main/1
Кейс-задача № 2	Для демонстрации работы базового и производного классов реализована объектно-ориентированная программа на Python на примере предметной области «транспортные средства». Базовый класс Vehicle описывает общие свойства и поведение (марка, максимальная скорость, методы start() и describe()), а производные классы Car и Truck наследуют его и расширяют собственными полями и методами. В производных классах продемонстрирован вызов конструктора базового класса через super().__init__(), что позволяет переиспользовать логику инициализации и не дублировать код. Метод describe() переопределён в каждом наследнике, при этом внутри переопределения через super().describe() вызывается базовая реализация, к которой добавляется специфичная информация — это наглядно показывает механизм расширения, а не полной замены поведения. Ключевым элементом является демонстрация полиморфизма: объекты разных классов помещаются в общий список типа Vehicle, и единый вызов describe() в цикле порождает разное поведение в зависимости от фактического типа

№ п/п	Подробные ответы обучающегося на практические кейсы-задачи
	объекта. Дополнительно у производных классов есть собственные методы (open_trunk у Car и load у Truck), недоступные базовому классу, что иллюстрирует специализацию. Корректность наследования, переопределения и полиморфизма подтверждена автоматическими тестами (test_task2.py), которые успешно проходят. Такая структура соответствует принципам ООП (инкапсуляция, наследование, полиморфизм) и легко расширяется новыми типами транспортных средств без изменения существующего кода. Ссылка на GitHub: https://github.com/StarcevAlexander/synergy-practice/tree/main/2
Кейс-задача № 3	Спроектирована реляционная база данных «Туризм» в виде SQL-скрипта для СУБД MySQL, совместимого с MySQL Workbench. База содержит пять таблиц: четыре таблицы-справочника (countries — страны/направления, hotels — отели, clients — клиенты, tours — туры как перечень предоставляемых услуг) и одну таблицу переменной информации (orders — заказы туров). Для каждой таблицы определён первичный ключ с автоинкрементом, что обеспечивает уникальную идентификацию записей. Связи между таблицами построены при помощи внешних ключей: таблица отелей ссылается на страны, таблица туров — на отели, а таблица переменной информации orders ссылается на ключевые атрибуты справочников clients и tours, что полностью соответствует требованию задания. В схему добавлены ограничения целостности (CHECK на количество звёзд, число ночей, число человек и неотрицательную цену), индексы по внешним ключам и дате для ускорения выборок, а также перечисляемый тип статуса заказа. Скрипт включает наполнение тестовыми данными и два примера аналитических запросов: расчёт полной стоимости каждого заказа с разворачиванием всех связей и агрегирование выручки по странам. Структура нормализована (приведена к 3НФ): справочные данные не дублируются, а переменная информация хранится отдельно и связывается ссылками. Такой подход исключает аномалии вставки, обновления и удаления и упрощает дальнейшее расширение модели (например, добавление таблиц оплат или менеджеров). Ссылка на GitHub: https://github.com/StarcevAlexander/synergy-practice/tree/main/3

№ п/п	Подробные ответы обучающегося на практические кейсы-задачи
Кейс-задача № 4	В рамках задачи проведён анализ представленных на рынке информационных систем WEB-архитектуры (CRM, ERP, системы бронирования, BI-системы, CMS-порталы) и описаны их возможности и варианты применения в компании — результаты сведены в документ ANALYSIS.md. WEB-архитектура реализует трёхзвенную модель «тонкий клиент (браузер) — сервер приложений — сервер базы данных», её преимущества — централизованное обновление, кроссплатформенность и масштабируемость, а основные риски связаны с безопасностью и зависимостью от сети. В задании предлагался стек Delphi 10.2 + IIS + MS SQL Server, однако, поскольку Delphi и IIS являются проприетарной Windows-средой, для воспроизводимости и кроссплатформенного запуска было разработано архитектурно эквивалентное WEB-приложение на Python/Flask, построенное по той же трёхзвенной схеме; при этом для целевой СУБД MS SQL Server подготовлена готовая схема базы данных (schema_mssql.sql) на языке T-SQL. Приложение на тему «Турагентство» реализует каталог туров, карточку отдельного тура, форму приёма заявок и страницу со списком заявок, а данные хранятся в базе (SQLite для локального запуска и MS SQL Server для целевого развёртывания). Серверная часть (роуты и доступ к БД) отделена от слоя представления (HTML-шаблоны с наследованием от базового шаблона), что соответствует принципу разделения ответственности. Корректность маршрутов проверена через тестовый клиент Flask, а для защиты от SQL-инъекций используются параметризованные запросы. Таким образом, требования задания по WEB-архитектуре и хранению данных в реляционной БД выполнены, а отступление от Delphi обосновано и компенсировано эквивалентной реализацией. Ссылка на GitHub: https://github.com/StarcevAlexander/synergy-practice/tree/main/4
Кейс-задача № 5	Для аналитического обзора выбрана работа из кейс-задачи № 4 (WEB-приложение «Турагентство»), оценённая по девяти критериям из индивидуального задания — полный текст приведён в файле analytical_review.md. По критерию функциональности приложение получает высокую оценку: все ключевые сценарии (просмотр каталога, оформление заявки, просмотр заявок) работают корректно и проверены. Производительность достаточна для учебного объёма данных, однако узким местом при росте нагрузки станет файловая СУБД SQLite. Юзабилити обеспечено простым адаптивным интерфейсом и понятной навигацией, но обработка ошибок в интерфейсе пока минимальна. По критерию безопасности отмечены параметризованные запросы (защита от SQL-инъекций), но выявлено отсутствие CSRF-защиты, аутентификации и HTTPS. Масштабируемость и переносимость оцениваются положительно

№ п/п	Подробные ответы обучающегося на практические кейсы-задачи
	благодаря трёхзвенной архитектуре, абстрагированию слоя данных и кроссплатформенности Python. Качество кода и сопровождаемость поддержаны разделением логики и представления, комментариями и типовыми аннотациями, а тестирование частично закрыто автотестами задач № 1 и № 2. На основе анализа предложены способы решения выявленных проблем: добавление автотестов роутов через тестовый клиент, страницы-обработчики ошибок и flash-сообщения, подключение CSRF-защиты (Flask-WTF) и аутентификации (Flask-Login), переход на MS SQL Server с пулом соединений, внедрение миграций схемы (Alembic/Flask-Migrate) и развёртывание за reverse-proxy с TLS. Реализация перечисленных мер переводит учебный прототип в состояние, близкое к промышленной эксплуатации. Ссылка на GitHub: https://github.com/StarcevAlexander/synergy-practice/tree/main/5

Дата: «26» июля 2026 г.

(подпись)

Старцев А.В.
(Ф.И.О. обучающегося)